

Problem Set 0

Harvard SEAS - Fall 2026

Due: Sat 2026-09-08 (11:59PM)

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.' Please remember to select pages when you submit on Gradescope. A problem set on the border between two letter grades cannot be rounded up if pages are not selected OR collaborators not noted.

Your name:

Collaborators and External Resources:

No. of late days used on previous psets:

No. of late days used after including this pset:

The purpose of this problem set is to reactivate your skills in proofs and programming from CS20 and CS32/CS50. For those of you who haven't taken one or both those courses, the problem set can also help you assess whether you have acquired sufficient skills to enter CS1200 in other ways and can fill in any missing gaps through self-study. Even for students with all of the recommended background, this problem set may still require a significant amount of thought and effort, so do not be discouraged if that is the case and do take advantage of the staff support in section and office hours.

For those of you who are wondering whether you should wait and take CS20 before taking CS1200, we encourage you to also complete [the CS20 Placement Self-Assessment](#). Some problems there that are of particular relevance to CS1200 are Problems 2 (counting), 4 & 5 (comparing growth rates), 9 (quantificational logic), and 12 (graph theory).

Written answers must be submitted in pdf format on Gradescope. Although L^AT_EX is not required, it is strongly encouraged. You may handwrite solutions so long as they are fully legible. The `ps0` directory, which contains your code for problems 1a and 1c, must be submitted separately to an autograder on Gradescope.

To obtain the starter code for the programming problems, please download it from [GitHub](#).

1. (Binary Trees)

In the [cs1200 GitHub repository](#), we have given you a Python implementation of a binary tree data structure, as well as a collection of test trees built using this data structure. We

specify a binary tree by giving a pointer to its *root*, which is a special *vertex* (a.k.a. *node*), and giving every vertex pointers to its *children* vertices and its *parent* vertex as well as an identifying *key*:

```
class BinaryTree:
    def __init__(self, root):
        self.root: BTvertex = root

class BTvertex:
    def __init__(self, key):
        self.parent: BTvertex = None
        self.left: BTvertex = None
        self.right: BTvertex = None
        self.key: int = key
        self.size: int = None
```

In CS50, the concept of a Python `class` was not covered. Here, with `BinaryTree` and `BTvertex`, we are using them in the same way as a `struct` in C. An object `v` of the `BTvertex` class contains five attributes, which we list with the type of the object we expect to be named by each attribute (using the Python type annotation syntax). These attributes can be accessed as `v.parent`, `v.left`, `v.right`, `v.key`, and `v.size`. For example, `v.left.key` is the key associated with `v`'s left child. An object of the `BinaryTree` class contains only one attribute, which is the `BTvertex` object that is the root of our binary tree. You can create a `BinaryTree` object as follows:

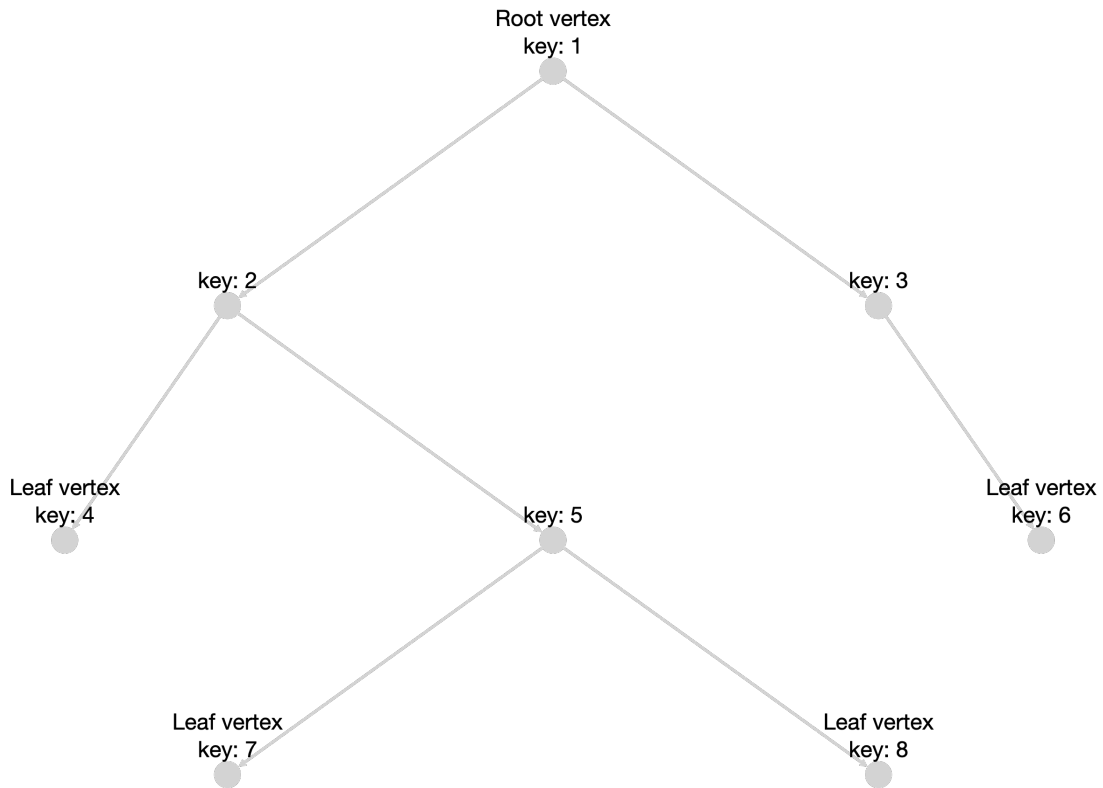
```
root = BTvertex(1200)
tree = BinaryTree(root)
tree.root.left = BTvertex(1210)
tree.root.right = BTvertex(1240)
```

You can then print attributes of the newly created `BinaryTree` object:

```
print(tree.root.key)
>> 1200
print(tree.root.left.key)
>> 1210
```

Classes are more general than structs because they can also have private attributes and methods that operate on the attributes, allowing for object-oriented programming. However, you won't need that generality in this problem set.

Here is an instance `T` of `BinaryTree`:



A `BinaryTree` `T` contains only a pointer to its root vertex, `T.root`, which is required to satisfy `T.root.parent==None`. In the above example, the root is the vertex with key 1 (i.e. `T.root.key==1`). A binary tree vertex `v` can have zero, one, or two children, determined by which of `v.left` and `v.right` are equal to `None`. In the above example, the vertex `v` with key 3 has `v.left==None` but `v.right` is the vertex with key 6. A *leaf* is a vertex with zero children, i.e. `v.left==v.right==None`.

A vertex `w` is *descendant* of a vertex `v` if there is a sequence of vertices $v_0, v_1, \dots, v_k$, $k \in \mathbb{N}$ such that $v_0 = v$, $v_k = w$, and $v_i \in \{v_{i-1}.left, v_{i-1}.right\}$ for $i = 1, \dots, k$.¹ In the above example, the vertex with key 5 is a descendant of the root (with a path of length 2), but is not a descendant of the vertex with key 3. The sequence $v_0, v_1, \dots, v_k$ is called a *path* from `v` to `w` and k is the *distance* from `v` to `w`. Taking $k = 0$, we see that `v` is a descendant of itself.

The *vertex set* of a binary tree `T` consists of all of the descendants of `T.root`. The *size* of `T` is its number of vertices. The *height* of `T` is the largest distance from the root to a leaf. The above example has size 8 and height 3.

Given any vertex `v` in a tree, the *subtree* rooted at `v` consists of all of `v`'s descendants. Note that we can remove a subtree and turn it into a new tree `S` by setting `S.root=v` and `v.parent=None`.

For now, the `key` attribute serves to distinguish vertices from each other in our tests and help illustrate what the algorithms are doing. The `BTvertex` class also has a `size` attribute,

¹ $\mathbb{N}$ denotes the natural numbers $\{0, 1, 2, 3, \dots\}$. Since we are computer scientists, we start counting at 0.

which is initialized to `None` in all of the test instances; it will be filled in by the program you write in Part 1a.

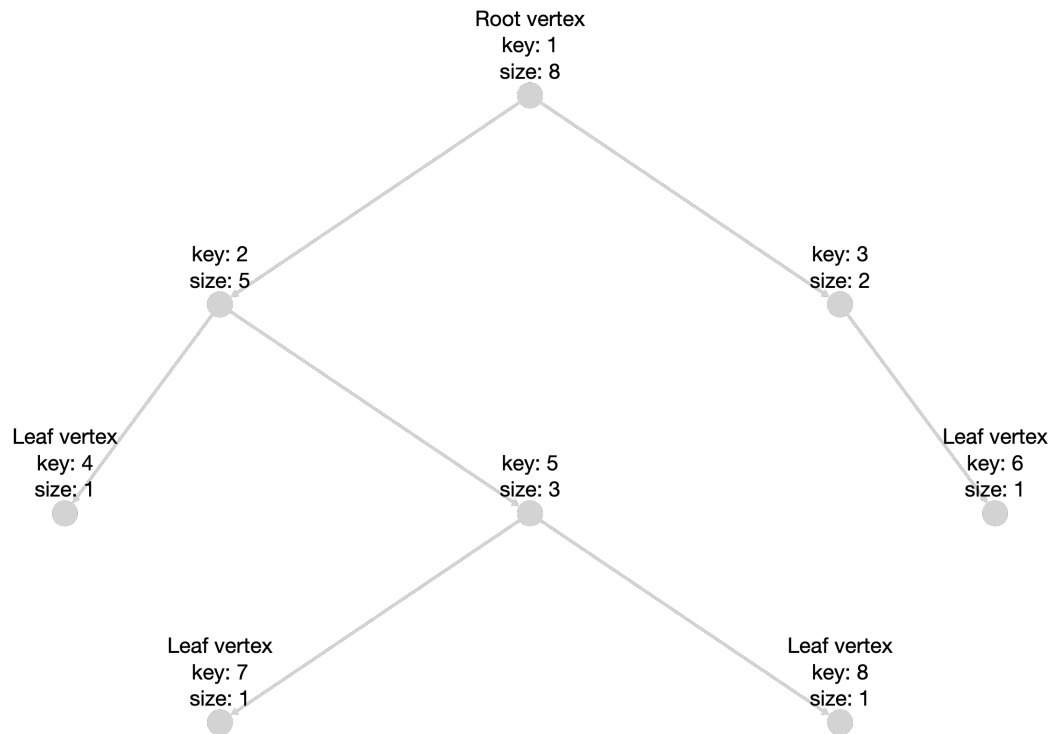
An instance `T BinaryTree` is *valid* if it satisfies the following constraints:

- `T.root.parent==None`
- `T` has finitely many vertices.
- No two vertices `v, w` of `T` share a child, i.e. $\{v.\text{left}, v.\text{right}\} \cap \{w.\text{left}, w.\text{right}\} = \emptyset$.

All of the test instances we provide are valid, and furthermore have the property that all of the vertices have distinct keys (which is something we often want, but not always).

- (a) (recursive programming) Write a recursive program `calculate_sizes` that given a vertex `v` of a binary tree `T`, calculates the sizes of all of the subtrees rooted at descendants of `v`. After running your program on `T.root`, every vertex `v` in `T` should have `v.size` set to the size of the subtree rooted at `v`. (Recall that the size attributes are initialized to `None`.) We call the resulting tree a *size-augmented tree*.

For example, if `T` is the tree shown above, then calling `calculate_sizes(T.root)` should modify `T` to be the following size-augmented tree:



Your program should run in time $O(n)$ when given the root of a tree with n vertices. In a sentence or two, informally justify why your program has such a runtime.

- (b) (proofs by induction) Let t be a positive integer that is fixed for this question. Prove that every binary tree `T` of size at least $2t + 1$ has a vertex `v` such that the subtree rooted at `v` has size at least t and at most $2t - 1$. (Hint: use strong induction on the size n of

- T, starting with base case $n = 2t + 1$. Remember that nodes in binary trees can have 2, 1, or 0 children.)
- (c) (from proofs to algorithms) Turn your proof from Part 1b into a Python program `FindDescendantOfSize(t,v)` that, given a positive integer t and a vertex v of a *size-augmented* tree T such that $v.size \geq 2t+1$, finds and returns a descendant w of v such that $t \leq w.size \leq 2t-1$. Your algorithm should run in time $O(h)$, where h is the height of the subtree rooted at v ; explain in words why this is the case.
2. (asymptotic notation) Recall the definitions of asymptotic notation from CS20: Let f and g be functions from $\mathbb{N}$ to $\mathbb{R}^+$, where $\mathbb{N} = \{0, 1, 2, \dots\}$ denotes the natural numbers (including 0, since we are computer scientists) and $\mathbb{R}^+$ denotes the positive real numbers.
- We say that $f = O(g)$ if there is a constant $c > 0$ such that $f(n) \leq c \cdot g(n)$ for all sufficiently large n .
 - We say that $f = o(g)$ if for *every* constant $c > 0$, $f(n) \leq c \cdot g(n)$ for all sufficiently large n ; equivalently, $\lim_{n \rightarrow \infty} (f(n)/g(n)) = 0$.
- (a) For each of the following pairs of functions, determine whether $f = O(g)$ and whether $f = o(g)$. Justify your answers.
- $f(n) = 3 \log_2^3 n$, $g(n) = n^2 + 1$.
 - $f(n) = |\{S \subseteq [n] : |S| \leq 5\}|$, where $[n] = \{0, 1, 2, \dots, n-1\}$, and $g(n) = 3n^3$.
 - $f(n) = 5^n$, $g(n) = n!$.
 - Prove or disprove: For all functions $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$, $f = O(g) \Rightarrow (g = o(f) \text{ OR } f = o(g))$.
3. (reflection: excitement and apprehension) On every problem set in cs1200, there will be a qualitative reflection question, usually about some aspect of your engagement with the course (but sometimes about the course content itself). Your answers to these questions, the in-class sender-receiver exercises and section attendance will be the basis of your participation grade. For starters, every thoughtful answer with some specifics will receive an R grade; R+ will be reserved for exceptional ones and grades below R for responses that are incomplete or generic.
- Quick note on grading:** Good responses are usually about a paragraph, with something like 7 or 8 sentences. Most importantly, please make sure your answer is specific to this class and your experiences in it. If your answer could have been edited lightly to apply to another class at Harvard, points will be taken off.
- Note: As with the previous psets, you may include your answer in your PDF submission, but the answer should ultimately go into a separate Gradescope submission form.*
- (Re)read the [syllabus](#). What aspect of cs1200 are you most excited about and what aspect are you most apprehensive about? Please address both excitement and apprehension in your answer, but it is also fine to say “I’m not excited about anything” or “I’m not apprehensive about anything.” What efforts can you make as a student to make the most out of your excitement, and to help address your apprehension?
4. Once you’re done with this problem set, please fill out [this survey](#) so that we can gather students’ thoughts on the problem set, and the class in general. It’s not required, but we really appreciate all responses!